Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

Lab Report 3
[Code No: COMP 342]

Submitted by:

Nischal Subedi (53)

Group: CS60 (III/I)

.

Submitted to:
Mr. Dhiraj Shrestha
Department of Computer Science and Engineering

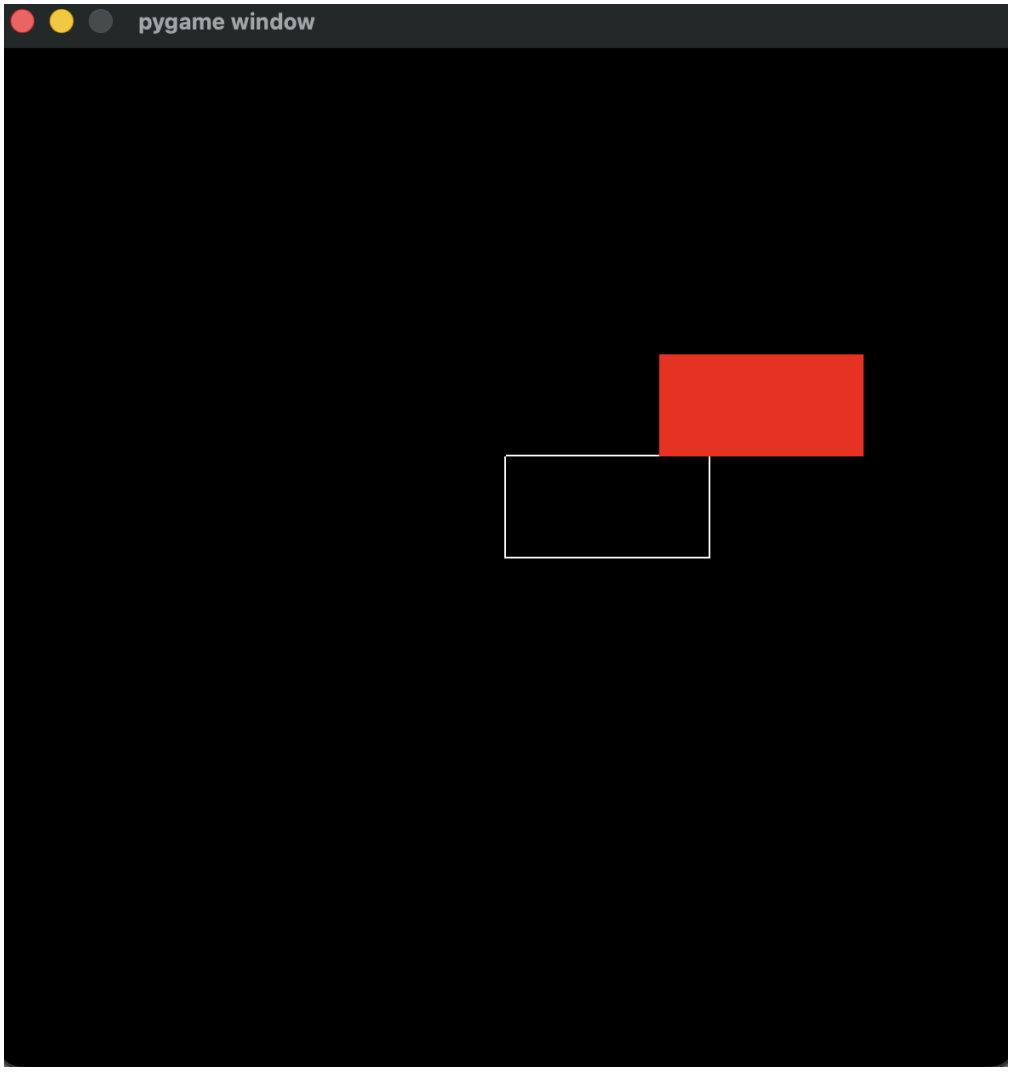
Submission Date: 04/01/2026

Q1.Write a program to implement 2D translation.

This program demonstrates moving a 2D object from one position to another. It utilizes a 3×3 homogeneous transformation matrix where the third column contains the translation offsets tx and ty. By multiplying the shape's vertices by this matrix, the object is shifted across the coordinate plane without changing its size or orientation.

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def translate_point(p, tx, ty):
    # Homogeneous Matrix for Translation
    matrix = [[1, 0, tx],
              [0, 1, ty],
              [0, 0, 1]]
    x, y = p
    # Matrix Multiplication: [1 0 tx][x], [0 1 ty][y], [0 0 1][1]
    new_x = matrix[0][0]*x + matrix[0][1]*y + matrix[0][2]*1
    new_y = matrix[1][0]*x + matrix[1][1]*y + matrix[1][2]*1
    return (new_x, new_y)

def main():
    pygame.init()
    pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
    gluOrtho2D(-10, 10, -10, 10)

    rect = [(0,0), (4,0), (4,2), (0,2)]
    transformed = [translate_point(p, 3, 2) for p in rect]

    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT: pygame.quit(); return
        glClear(GL_COLOR_BUFFER_BIT)
        # Original (White)
        glColor3f(1, 1, 1)
        glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in rect]; glEnd()
        # Transformed (Red)
        glColor3f(1, 0, 0)
```

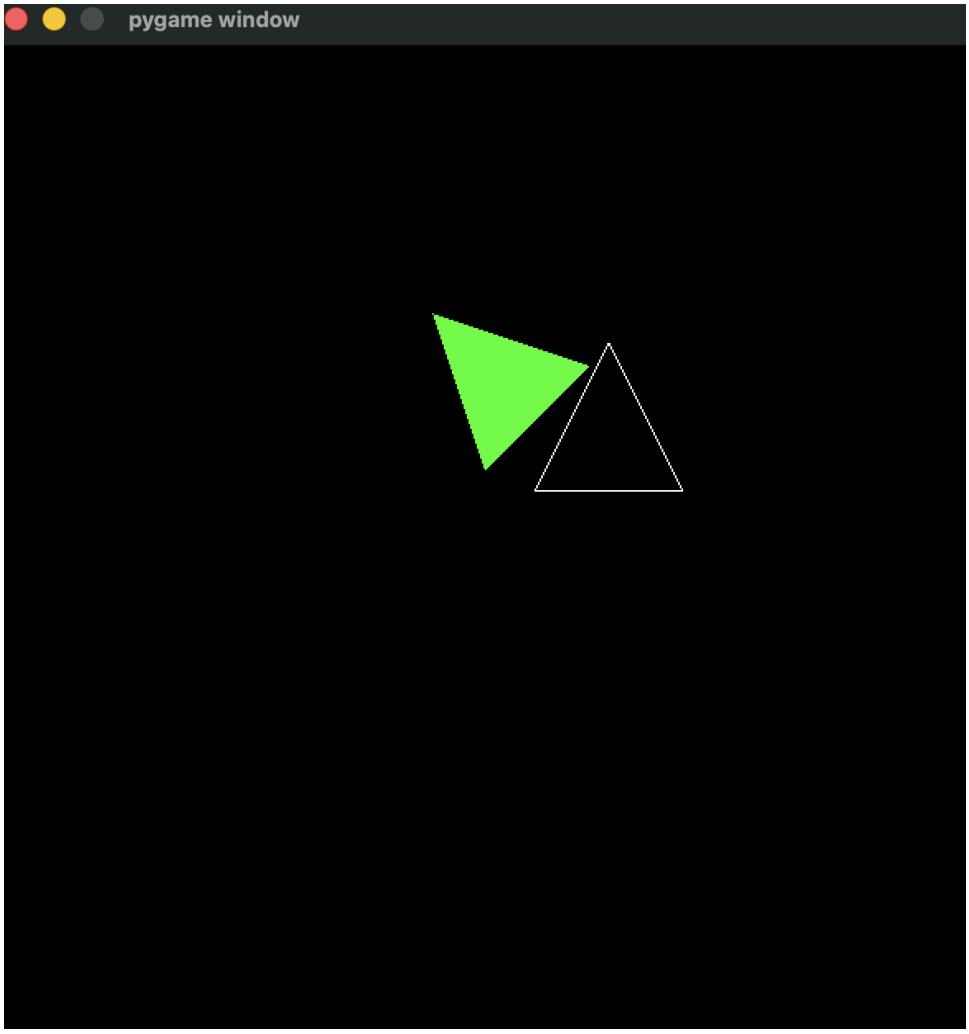
```
        glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in transformed]; glEnd()
        pygame.display.flip()

if __name__ == "__main__": main()
```



Q2. Write a program to implement 2D rotation

This script implements the rotation of a 2D shape around the origin (0,0). The transformation uses trigonometric functions (sin and cos) within a homogeneous matrix. The vertices are recalculated based on a specified angle θ, effectively swinging the object around the center of the coordinate system.

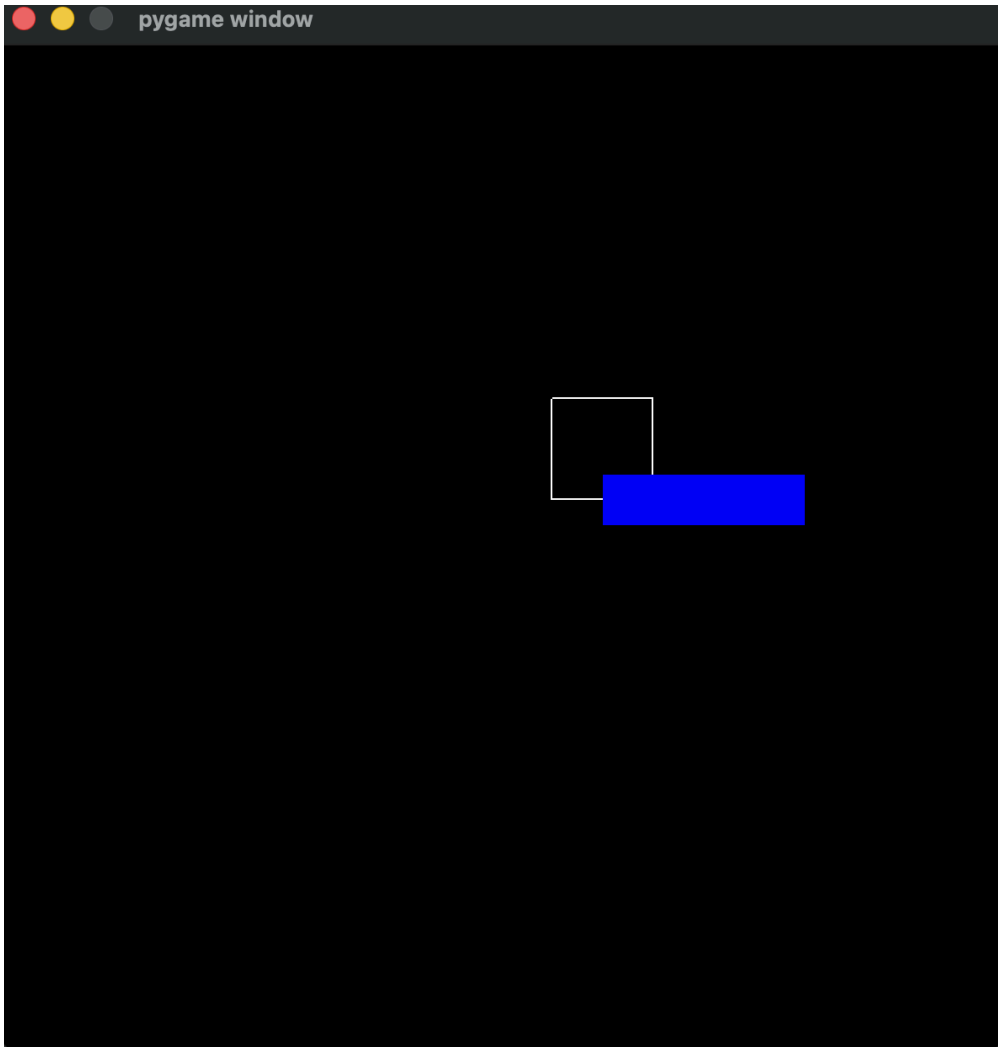
```
import pygame, math
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def rotate_point(p, angle):
   rad = math.radians(angle)
   c, s = math.cos(rad), math.sin(rad)
   # Homogeneous Rotation Matrix
   matrix = [[c, -s, 0], [s, c, 0], [0, 0, 1]]
   x, y = p
   new_x = matrix[0][0]*x + matrix[0][1]*y + matrix[0][2]*1
   new_y = matrix[1][0]*x + matrix[1][1]*y + matrix[1][2]*1
   return (new_x, new_y)

def main():
   pygame.init()
   pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
   gluOrtho2D(-10, 10, -10, 10)
   tri = [(1,1), (4,1), (2.5,4)]
   transformed = [rotate_point(p, 45) for p in tri]
   while True:
       for event in pygame.event.get():
           if event.type == pygame.QUIT: pygame.quit(); return
       glClear(GL_COLOR_BUFFER_BIT)
       glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
tri]; glEnd()
       glColor3f(0, 1, 0); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
       pygame.display.flip()

if __name__ == "__main__": main()
```



Q3.Write a program to implement 2D scaling.

This program performs uniform or non-uniform resizing of a 2D shape. The transformation matrix contains scaling factors sx and sy along the main diagonal. Values greater than **1** enlarge the shape, while values between **0** and **1** shrink it. Because the transformation is relative to the origin, the object also moves toward or away from (0,0) as it changes size.

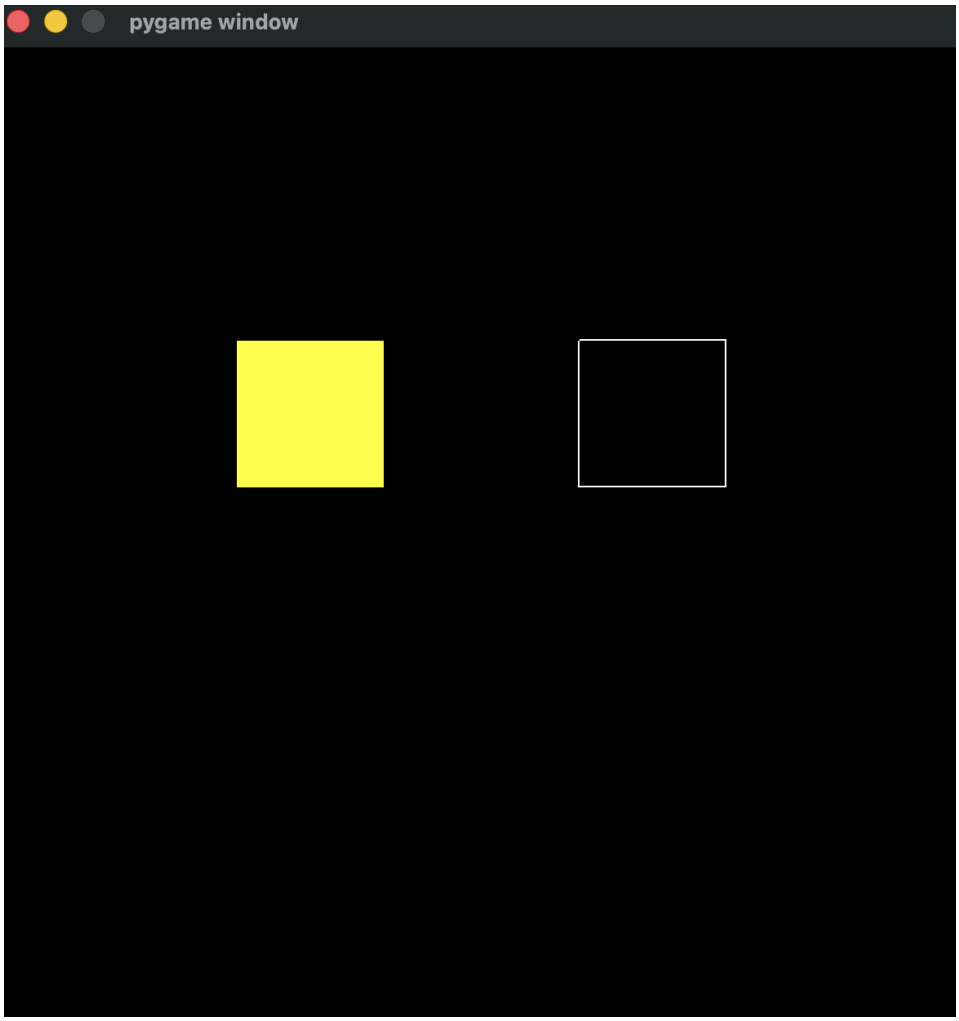
```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def scale_point(p, sx, sy):
   matrix = [[sx, 0, 0], [0, sy, 0], [0, 0, 1]]
   x, y = p
   return (matrix[0][0]*x, matrix[1][1]*y)

def main():
   pygame.init()
   pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
   gluOrtho2D(-10, 10, -10, 10)
   square = [(1,1), (3,1), (3,3), (1,3)]
   transformed = [scale_point(p, 2, 0.5) for p in square]
   while True:
       for event in pygame.event.get():
           if event.type == pygame.QUIT: pygame.quit(); return
       glClear(GL_COLOR_BUFFER_BIT)
       glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
square]; glEnd()
       glColor3f(0, 0, 1); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
       pygame.display.flip()

if __name__ == "__main__": main()
```



Q4. Write a program to implement 2D reflection

Reflection creates a mirror image of an object across a specific axis. This implementation uses a matrix that negates the x-coordinate (to reflect across the Y-axis) while keeping the y-coordinate constant. This effectively "flips" the geometry to the opposite side of the Cartesian plane.

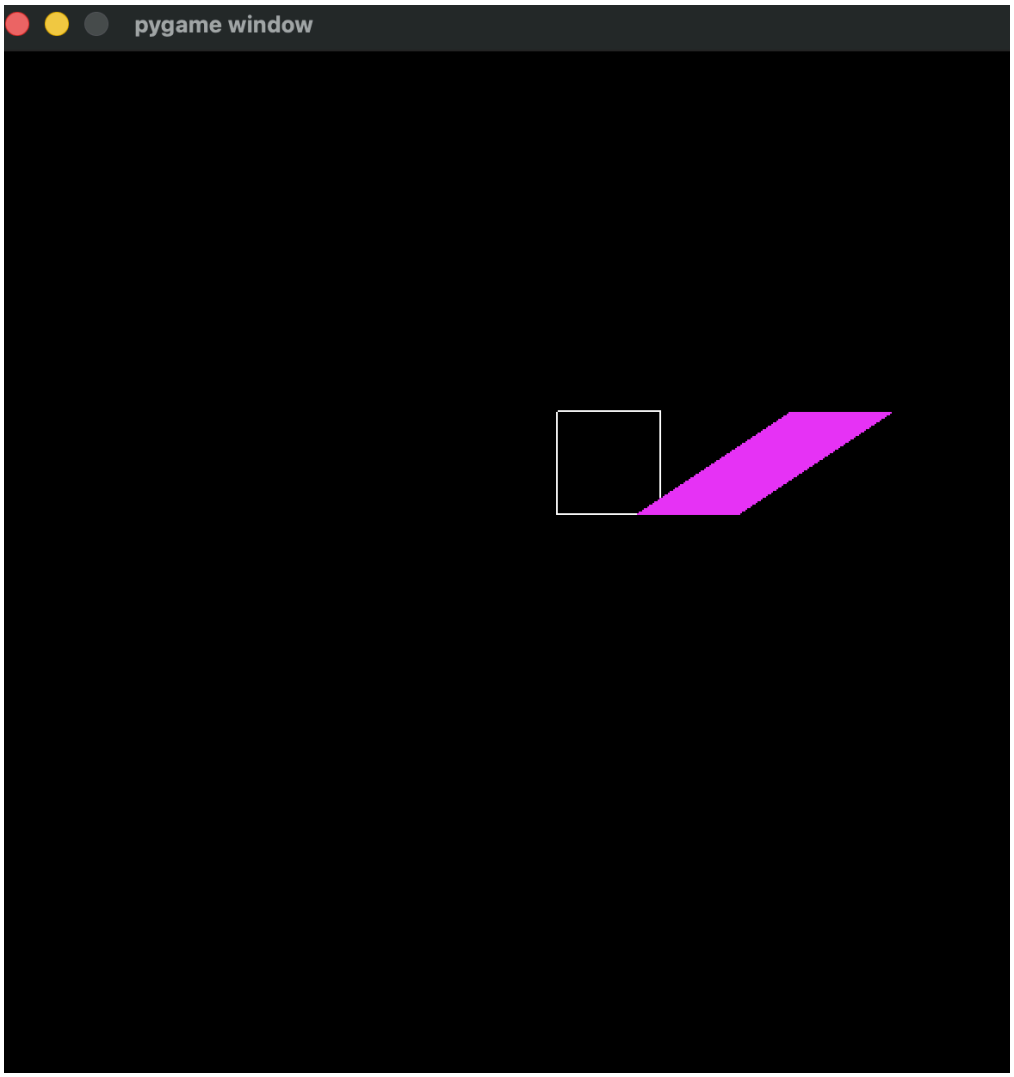
```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def reflect_point(p):
   # Reflection over Y-axis: x becomes -x
   matrix = [[-1, 0, 0], [0, 1, 0], [0, 0, 1]]
   x, y = p
   return (matrix[0][0]*x, matrix[1][1]*y)

def main():
   pygame.init()
   pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
   gluOrtho2D(-10, 10, -10, 10)
   shape = [(2,1), (5,1), (5,4), (2,4)]
   transformed = [reflect_point(p) for p in shape]
   while True:
       for event in pygame.event.get():
           if event.type == pygame.QUIT: pygame.quit(); return
       glClear(GL_COLOR_BUFFER_BIT)
       glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
shape]; glEnd()
       glColor3f(1, 1, 0); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
       pygame.display.flip()

if __name__ == "__main__": main()
```



Q5. Write a program to implement 2D shearing

Shearing (or skewing) distorts the shape of an object such that it appears to be tilted. The program uses a shear matrix where the off-diagonal elements shx or shy are non-zero. This causes the coordinates of one axis to be modified based on the current value of the other axis, turning a rectangle into a parallelogram.

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def shear_point(p, shx, shy):
   matrix = [[1, shx, 0], [shy, 1, 0], [0, 0, 1]]
   x, y = p
   new_x = x + shx * y
   new_y = shy * x + y
   return (new_x, new_y)

def main():
   pygame.init()
   pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
   gluOrtho2D(-10, 10, -10, 10)
   square = [(1,1), (3,1), (3,3), (1,3)]
   transformed = [shear_point(p, 1.5, 0) for p in square]
   while True:
       for event in pygame.event.get():
           if event.type == pygame.QUIT: pygame.quit(); return
       glClear(GL_COLOR_BUFFER_BIT)
       glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
square]; glEnd()
       glColor3f(1, 0, 1); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
       pygame.display.flip()

if __name__ == "__main__": main()
```



Q6. Write a program to implement composite transformations

This program demonstrates the power of the homogeneous coordinate system by combining four distinct transformations—**Scaling, Rotation, Shearing, and Translation**—into a single operation. By manually performing matrix multiplication ($T \cdot Sh \cdot R \cdot S$), we derive a single "Global Transformation Matrix." This allows all four steps to be applied to the vertices in a single mathematical pass, ensuring high efficiency.

```
import pygame, math
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def multiply_3x3(A, B):
    C = [[0,0,0], [0,0,0], [0,0,0]]
    for i in range(3):
        for j in range(3):
            for k in range(3):
                C[i][j] += A[i][k] * B[k][j]
    return C

def main():
    pygame.init()
    pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
    gluOrtho2D(-10, 10, -10, 10)

    # 1. Scaling Matrix
    S = [[0.5, 0, 0], [0, 0.5, 0], [0, 0, 1]]
    # 2. Rotation Matrix (30 deg)
    a = math.radians(30); c, s = math.cos(a), math.sin(a)
    R = [[c, -s, 0], [s, c, 0], [0, 0, 1]]
    # 3. Shear Matrix
    Sh = [[1, 0.5, 0], [0, 1, 0], [0, 0, 1]]
    # 4. Translation Matrix
    T = [[1, 0, 2], [0, 1, -3], [0, 0, 1]]

    # Composite = T * Sh * R * S (Applied right to left)
    comp = multiply_3x3(T, multiply_3x3(Sh, multiply_3x3(R, S)))

    shape = [(0,0), (2,0), (2,2), (0,2)]
    transformed = []
    for x, y in shape:
        nx = comp[0][0]*x + comp[0][1]*y + comp[0][2]*1
        ny = comp[1][0]*x + comp[1][1]*y + comp[1][2]*1
        transformed.append((nx, ny))

    while True:
```

```
        for event in pygame.event.get():
            if event.type == pygame.QUIT: pygame.quit(); return
        glClear(GL_COLOR_BUFFER_BIT)
        glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
shape]; glEnd()
        glColor3f(0, 1, 1); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
        pygame.display.flip()

if __name__ == "__main__": main()
```